

# Jobsheet Praktikum: Pertemuan 2

## Kelas dan Objek

|                    |                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Mata Kuliah</b> | Praktikum Pemrograman Berbasis Objek (RTI253008)                                                                      |
| <b>Pertemuan</b>   | 2 (Minggu 2)                                                                                                          |
| <b>Durasi</b>      | 1 × 4 × 50' praktikum; 1 × 1 × 50' tugas/laporan mandiri                                                              |
| <b>Sub-CPMK</b>    | SCPMK0210-02601: mengimplementasikan kelas, enkapsulasi, relasi kelas, dan inheritance dalam kode Java yang berfungsi |
| <b>Kode Awal</b>   | tidak ada, kita mulai dari proyek kosong                                                                              |
| <b>Kode Akhir</b>  | proyek oop-praktikum setelah Langkah 7                                                                                |

## A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, kamu mampu:

1. Mendefinisikan kelas Java dengan atribut, konstruktor, dan method.
2. Membuat dan memanipulasi objek menggunakan new, serta memahami perilaku referensi.
3. Menerjemahkan diagram kelas UML sederhana menjadi kode Java.

## B. Persiapan dan Prasyarat

- **Alat:** JDK 17 atau lebih baru, NetBeans (editor yang kita pakai sepanjang praktikum ini).
- **Proyek:** hari ini kamu bikin proyek baru bernama oop-praktikum. Semua kelas ditaruh dalam package `id.ac.polinema` (ini konvensi umum penamaan package di Java: nama domain institusi dibalik urutannya, "polinema.ac.id" jadi `id.ac.polinema`).
- **Verifikasi cepat** sebelum mulai:

```
java -version
javac -version
```

Kalau keduanya menampilkan nomor versi tanpa error, kamu siap lanjut.

**Tanpa NetBeans?** Nggak masalah, kamu tetap bisa ikuti jobsheet ini pakai editor teks biasa:

```
mkdir -p oop-praktikum/src/id/ac/polinema
cd oop-praktikum
javac -d out src/id/ac/polinema/*.java
java -cp out id.ac.polinema.Main
```

Checkpoint dan outputnya sama persis, apa pun editor yang kamu pakai.

## C. Langkah Kerja

### Langkah 1: Buat proyek

Buka NetBeans, buat project baru tipe **Java Application**, kasih nama oop-praktikum. Waktu NetBeans minta nama package untuk kelas utama, isi dengan `id.ac.polinema`. Struktur folder dan file `Main.java` akan dibuatkan otomatis di dalam package itu.

- ✓ **Checkpoint:** panel Projects menampilkan oop-praktikum dengan package `id.ac.polinema` berisi `Main.java`.

### Langkah 2: Kelas `Rectangle` minimal dan objek pertama

**Kelas** itu semacam cetakan: dia menjelaskan atribut dan perilaku apa yang bakal dimiliki tiap **objek** yang dibuat darinya. Kelas `Rectangle` di bawah ini baru punya atribut (state), belum ada method sama sekali.

Tambahkan kelas baru Rectangle ke package id.ac.polinema (klik kanan package > New > Java Class):

```
1 package id.ac.polinema;
2
3 public class Rectangle {
4     int width;
5     int height;
6 }
```

Figure 1: Rectangle.java: kelas dengan dua atribut

Ganti isi Main.java untuk membuat satu objek Rectangle dan mengisi atributnya satu per satu:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle r = new Rectangle();
6         r.width = 6;
7         r.height = 4;
8
9         System.out.println("Rectangle " + r.width + "x" + r.height);
10    }
11 }
```

Figure 2: Main.java: membuat objek Rectangle dan mengisi atribut secara manual

Jalankan (klik kanan proyek > Run, atau tombol F6). Kalau kamu pakai terminal: `javac -d out src/id/ac/polinema/*.java && java -cp out id.ac.polinema.Main`.

✓ **Checkpoint:** program menampilkan Rectangle 6x4.

⚠ **Jika gagal:** kalau muncul error: cannot find symbol, cek dulu apakah nama berkas sama persis dengan nama kelasnya (Rectangle.java untuk class Rectangle). Java itu case-sensitive, jadi huruf besar-kecil harus pas.

### Langkah 3: Menambahkan method (perilaku)

Atribut doang cuma nyimpen data, nggak ngapa-ngapain. **Method** itu yang bikin kelas punya perilaku, sesuatu yang bisa dilakukan objek dengan datanya sendiri. Yuk tambahkan dua method ke Rectangle (baris hijau menandai kode yang baru ditambahkan):

Sekarang sederhanakan Main.java supaya memakai method barusan:

Jalankan ulang.

✓ **Checkpoint:** output-nya jadi dua baris, Area: 24 lalu Perimeter: 20.

⚠ **Jika gagal:** kalau muncul galat non-static method area() cannot be referenced from a static context, berarti kamu panggil area() tanpa lewat objek (misalnya Rectangle.area()). Method non-static itu wajib dipanggil lewat objeknya, jadi harus r.area().

### Langkah 4: Konstruktor dan this

Sadar nggak, ada jeda waktu antara new Rectangle() dan atributnya selesai diisi (lihat Langkah 2-3)? Selama jeda itu, objeknya masih “setengah jadi”, atributnya masih kosong (0 untuk int). **Konstruktor** menutup celah ini dengan mewajibkan data lengkap begitu objek dibuat.

```

1 package id.ac.polinema;
2
3 public class Rectangle {
4     int width;
5     int height;
6
7     int area() {
8         return width * height;
9     }
10
11     int perimeter() {
12         return 2 * (width + height);
13     }
14 }

```

Figure 3: Rectangle.java dengan method area() dan perimeter() ditambahkan

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle r = new Rectangle();
6         r.width = 6;
7         r.height = 4;
8
9         System.out.println("Area: " + r.area());
10        System.out.println("Perimeter: " + r.perimeter());
11    }
12 }

```

Figure 4: Main.java memakai area() dan perimeter()

```

1 package id.ac.polinema;
2
3 public class Rectangle {
4     int width;
5     int height;
6
7     Rectangle(int width, int height) {
8         this.width = width;
9         this.height = height;
10    }
11
12    int area() {
13        return width * height;
14    }
15
16    int perimeter() {
17        return 2 * (width + height);
18    }
19 }

```

Figure 5: Rectangle.java dengan konstruktor ditambahkan

this.width merujuk ke atribut milik objek, sedangkan width di sisi kanan itu parameter konstruktor. Karena namanya sengaja dibikin sama, kita butuh this biar Java tahu mana yang kita maksud.

Sederhanakan Main.java:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle r = new Rectangle(6, 4);
6
7         System.out.println("Area: " + r.area());
8         System.out.println("Perimeter: " + r.perimeter());
9     }
10 }
```

Figure 6: Main.java memakai konstruktor Rectangle

✓ **Checkpoint:** output sama persis dengan Langkah 3, tapi kode Main.java-mu jauh lebih pendek sekarang.

Nah, ini bagian yang sering bikin bingung: begitu kamu menulis `new Rectangle(6, 4)`, sebenarnya ada dua hal yang kejadian di memori. Variabel `r` sendiri cuma tempat kecil di **stack**, isinya cuma alamat. Objek `Rectangle` yang sebenarnya, lengkap dengan `width` dan `height`-nya, hidup terpisah di **heap**. Gambarnya begini:

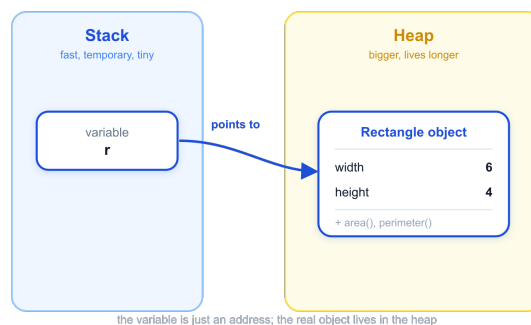


Figure 7: Variabel di stack menunjuk ke objek Rectangle di heap

⚠ **Jika gagal:** galat constructor `Rectangle` in class `Rectangle` cannot be applied to given types biasanya berarti jumlah atau urutan argumen di `new Rectangle(...)` nggak cocok sama parameter konstruktornya. Hitung ulang, ya.

### Langkah 5: Referensi, aliasing, dan null

Variabel objek di Java itu bukan objeknya sendiri, tapi **referensi** yang menunjuk ke objek di memori. Karena itu, dua variabel bisa saja menunjuk ke objek yang persis sama. Tambahkan kode berikut ke akhir main (baris hijau):

Jalankan.

✓ **Checkpoint:** dua baris pertama (Via original: dan Via copy:) sama-sama menampilkan 40, padahal kamu cuma mengubah `copy.width`. Ini bukan bug, ini emang begitu cara kerjanya: original dan copy menunjuk ke objek yang sama persis, jadi mengubah lewat salah satu variabel otomatis “terlihat” dari variabel lainnya. Baris ketiga bakal gagal dengan `NullPointerException`, dan ini memang disengaja.

Begitu gambarnya kalau digambar di stack dan heap:

Sekarang perbaiki: hapus dua baris terakhir (`Rectangle empty = null;` dan pemanggilan `area()` di atasnya) supaya program jalan lagi tanpa error:

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle original = new Rectangle(6, 4);
6
7         System.out.println("Area: " + original.area());
8
9         Rectangle copy = original;
10        copy.width = 10;
11        System.out.println("Via original: " + original.area());
12        System.out.println("Via copy: " + copy.area());
13
14        Rectangle empty = null;
15        System.out.println(empty.area());
16    }
17 }

```

Figure 8: Main.java dengan blok aliasing dan uji null ditambahkan

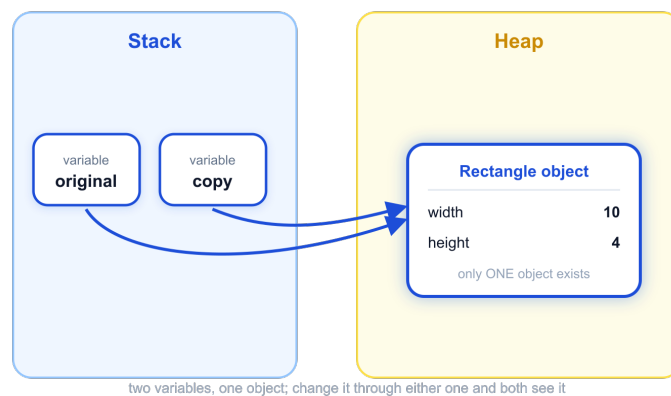


Figure 9: Dua variabel di stack menunjuk ke satu objek Rectangle yang sama di heap

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle original = new Rectangle(6, 4);
6
7         System.out.println("Area: " + original.area());
8
9         Rectangle copy = original;
10        copy.width = 10;
11        System.out.println("Via original: " + original.area());
12        System.out.println("Via copy: " + copy.area());
13    }
14 }

```

Figure 10: Main.java setelah dua baris uji null dihapus

⚠ **Jika gagal:** NullPointerException selalu muncul kalau kamu memanggil method pada referensi yang belum menunjuk ke objek mana pun (null). Solusinya selalu sama: pastikan objeknya sudah benar-benar dibuat dengan new sebelum method-nya dipanggil.

## Langkah 6: Kelas Student dari diagram UML

Diagram kelas UML berikut menggambarkan kelas Student yang perlu kamu buat:

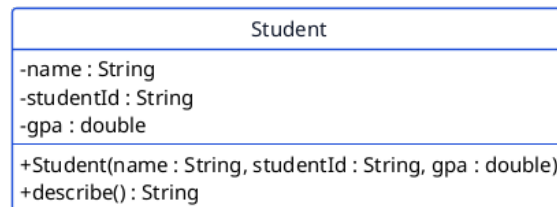


Figure 11: Diagram kelas UML untuk Student

Tanda - di depan atribut berarti privat (cuma bisa diakses dari dalam kelas itu sendiri), tanda + berarti method-nya publik. Buat kelas baru Student:

```
1 package id.ac.polinema;
2
3 public class Student {
4     private String name;
5     private String studentId;
6     private double gpa;
7
8     Student(String name, String studentId, double gpa) {
9         this.name = name;
10        this.studentId = studentId;
11        this.gpa = gpa;
12    }
13
14    public String describe() {
15        return name + " (" + studentId + ", GPA: " + gpa + ")";
16    }
17 }
```

Figure 12: Student.java sesuai diagram UML

Tambahkan satu baris pengujian di akhir main pada Main.java:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle original = new Rectangle(6, 4);
6
7         System.out.println("Area: " + original.area());
8
9         Rectangle copy = original;
10        copy.width = 10;
11        System.out.println("Via original: " + original.area());
12        System.out.println("Via copy: " + copy.area());
13
14        Student s = new Student("Nadia", "S001", 3.8);
15        System.out.println(s.describe());
16    }
17 }
```

Figure 13: Main.java dengan pengujian Student ditambahkan

✓ **Checkpoint:** program berhasil dikompilasi ulang dan baris terakhir menampilkan Nadia (S001, GPA: 3.8).

## Langkah 7: Array of objects, banyak objek dari satu kelas

Satu kelas bisa menghasilkan banyak objek sekaligus. Ganti bagian pengujian Rectangle (Langkah 4-5) dengan array Rectangle[], tapi tetap pertahankan bagian Student dari Langkah 6:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Rectangle[] shapes = new Rectangle[3];
6         shapes[0] = new Rectangle(6, 4);
7         shapes[1] = new Rectangle(3, 3);
8         shapes[2] = new Rectangle(8, 2);
9
10        for (Rectangle r : shapes) {
11            System.out.println("Area: " + r.area() + ", Perimeter: " + r.perimeter());
12        }
13
14        Student s = new Student("Nadia", "S001", 3.8);
15        System.out.println(s.describe());
16    }
17 }
```

Figure 14: Main.java final: array Rectangle[] dan pengujian Student

✓ **Checkpoint:** program mencetak tiga baris area/perimeter (satu per elemen array, nilainya beda-beda karena tiap Rectangle punya ukurannya sendiri), diikuti baris Nadia (S001, GPA: 3.8).

⚠ **Jika gagal:** ArrayIndexOutOfBoundsException artinya kamu mengisi indeks yang nggak ada, misalnya shapes[3] padahal array-nya cuma berukuran 3 (indeks yang valid: 0, 1, 2).

## D. Tugas dan Deliverable

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 7.
- **Tugas mandiri:**
  1. Buat kelas Circle sesuai diagram UML berikut:

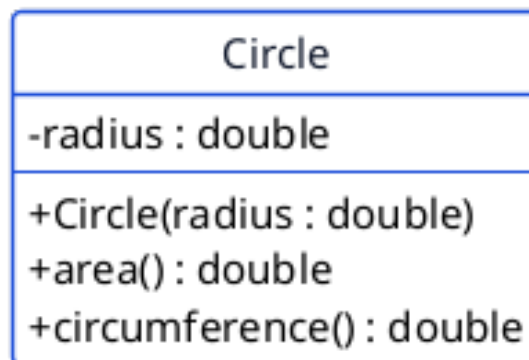


Figure 15: Diagram kelas UML untuk Circle

Method area() dan circumference() mengembalikan double, hitung pakai rumus lingkaran biasa ( $\text{Math.PI} * \text{radius} * \text{radius}$  untuk luas,  $2 * \text{Math.PI} * \text{radius}$  untuk keliling). Buktikan dengan membuat satu objek Circle di Main (radius 5) dan mencetak kedua hasilnya.

2. Jawab singkat (2-3 kalimat masing-masing): (a) apa bedanya objek dengan referensi ke objek? (b) tepatnya kapan konstruktor sebuah kelas dijalankan?

## E. Kriteria Penilaian

| Komponen                 | Bobot | Kriteria Lengkap (100%)                                      | Kriteria Minimum                                     |
|--------------------------|-------|--------------------------------------------------------------|------------------------------------------------------|
| Langkah kerja tuntas     | 40%   | Seluruh langkah dijalankan dan berfungsi                     | Sebagian besar langkah selesai, hasil akhir berjalan |
| Checkpoint terverifikasi | 35%   | Semua checkpoint tercapai dan dibuktikan (screenshot/output) | Sebagian checkpoint terbukti                         |
| Tugas mandiri            | 25%   | Kelas Circle benar dan jawaban konsep tepat                  | Kelas Circle ada meski jawaban belum lengkap         |